

Tarea 7 — IA Generativa

Explicación del funcionamiento del código

Skills, RAG industrial, chatbot del curso, agente de mantenimiento y LLMs en microcontroladores

Estudiante: Juan Diego Ruiz

Curso: IA y Minirobots — Universidad Nacional de Colombia

Guía de referencia: 7. Inteligencia Artificial Generativa (J. J. Martínez P., sept. 2026)

Fecha: 6 de octubre de 2026

1. Visión general

Este documento explica cómo funciona el código entregado en el notebook Tarea7.ipynb y cómo cada ejercicio se conecta con los conceptos de la guía 7. Los ejercicios 2, 3 y 4 se ejecutan localmente con **Ollama** (fuera de Colab), por lo que el código completo, los PDF y el archivo requirements.txt están en el repositorio github.com/Jruiztr28/IA-y-Minirobots, carpeta Tarea 7.

Ejercicio	Qué pide la guía	Concepto de la guía	Tecnología usada
1	Definir 4 skills propias	Skills (§7.11), Agentes (§7.8)	Archivos SKILL.md (Markdown + frontmatter YAML)
2	RAG industrial con manuales de máquinas	RAG (§7.7), embeddings y tokens (§7.5)	pypdf, numpy, Ollama: nomic-embed-text + qwen2.5:7b
3	Chatbot con los documentos del curso	RAG (§7.7)	Mismo pipeline del ejercicio 2
4	Agente de mantenimiento con herramientas	Agentes y herramientas (§7.8)	sqlite3 + tool calling de Ollama (qwen2.5:7b)
5	¿Un ESP32 puede usar un LLM?	Costo de tokens, tamaño de modelos (§7.4–7.5)	Respuesta teórica

2. Ejercicio 1 — Skills

Según la guía, un skill es un paquete de instrucciones anclado por un archivo Markdown llamado SKILL.md. El notebook primero crea las cuatro carpetas con `mkdir -p` y luego escribe cada archivo con la magia de Jupyter `%writefile`, que guarda el contenido de la celda directamente en disco.

Estructura de cada SKILL.md

Todos los skills siguen la misma estructura en tres partes:

- **Frontmatter YAML** (entre `---`): `name` identifica el skill, `description` le dice al modelo qué hace y cuándo usarlo, y `disable-model-invocation: true` hace que el skill solo se ejecute cuando el usuario lo llama explícitamente (el modelo no lo activa por su cuenta).
- **Proceso y reglas**: pasos numerados que el modelo debe seguir, más restricciones. Una regla común a los cuatro es *no inventar información y preguntar al usuario si falta algo*, lo cual ataca directamente el problema de las alucinaciones que describe la guía.
- **Plantilla** (entre `<plantilla>`): el formato exacto de la salida, para que el resultado sea consistente cada vez que se use el skill.

```

---
name: ticket-work
description: Documenta un ticket de trabajo por hacer en un ticket completo...
disable-model-invocation: true
---
# Documentar Ticket
## Proceso
1. Lee todo lo que el usuario escribió.
2. No inventes datos, consulta si falta información.
...
<plantilla> ... </plantilla>

```

Los cuatro skills y cómo se encadenan

Skill	Entrada	Qué hace	Salida
captura-work	Documentos del usuario	Extrae requerimientos, los clasifica en funcionales / no funcionales y cita la fuente (documento y página). Lo ambiguo va a una sección de dudas.	Tabla de requerimientos + dudas
ticket-work	Descripción del trabajo	Rellena un ticket con contexto, objetivo, alcance, tareas, criterios de aceptación, dependencias, riesgos y prioridad.	Ticket completo
Gestion-work	Ticket	Implementa cada criterio con ciclo TDD: prueba que falla → código que la pasa → refactorización. Mínimo 3 pruebas por criterio.	Código, pruebas y reporte
explicación-work	Ticket + archivos creados	Genera un HTML local (sin internet) en lenguaje sencillo, con diagrama de flujo, cómo probarlo y limitaciones.	Archivo .html

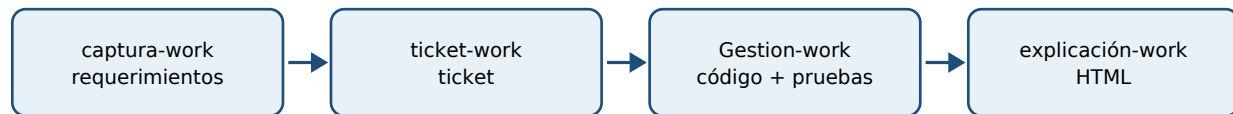


Figura 1. Los skills forman un flujo de trabajo completo: de los documentos al código documentado.

3. Ejercicio 2 — RAG industrial

El programa es un chatbot que responde preguntas sobre manuales técnicos de máquinas (carpeta manuales/). Implementa la arquitectura RAG de la sección 7.7 de la guía: en lugar de confiar solo en el conocimiento del LLM, primero **recupera** los fragmentos de los manuales más relacionados con la pregunta y luego se los entrega al LLM para que **genere** la respuesta.

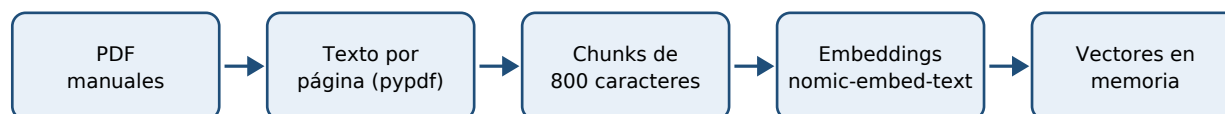


Figura 2a. Preparación de datos (se ejecuta una sola vez al iniciar).

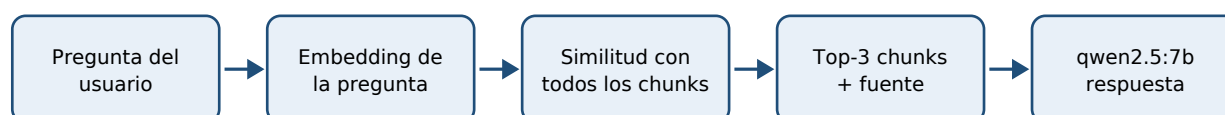


Figura 2b. Ciclo de consulta (se repite por cada pregunta).

Relación con los pasos de la guía

Paso de la guía (§7.7)	Función del código
Paso 2 — Extracción de información y división en chunks	leer_manuales() y cortar_texto()
Paso 3 — Obtención de vectores embedding	crear_embeddings()
Paso 4 — Consulta del usuario convertida en embedding	buscar() (primera parte)
Paso 5 — Recuperación de los chunks más cercanos	buscar() (similitud y top-3)
Paso 6 — Respuesta del LLM	preguntar_a_qwen()

3.1 Lectura y fragmentación: leer_manuales() y cortar_texto()

`leer_manuales()` recorre todos los .pdf de la carpeta y, con `PdfReader`, extrae el texto de cada página. El texto de cada página se pasa a `cortar_texto()`, que lo divide en trozos de 800 caracteres y descarta los vacíos. Se cortan por página (y no el documento completo) para que cada chunk quede asociado a su origen: en la lista paralela `fuentes` se guarda "`archivo.pdf, página N`", lo que luego permite al chatbot citar de dónde sacó la información.

```
for numero, pagina in enumerate(pdf.pages, start=1):
    texto = pagina.extract_text() or ""
    for parte in cortar_texto(texto):
        textos.append(parte)
        fuentes.append(f"{archivo.name}, página {numero}")
```

Los chunks son necesarios por el límite de la ventana de contexto que menciona la guía: no se puede enviar un manual entero al modelo en cada pregunta, así que se envían solo los trozos relevantes.

3.2 Embeddings: crear_embeddings()

Cada chunk se convierte en un vector numérico con el modelo **nomic-embed-text** a través de `ollama.embed()`. Internamente el modelo tokeniza el texto y produce un vector que representa su significado, tal como en el ejemplo Perro/Gato/Lobo/Robot de la guía, pero con cientos de dimensiones. Los chunks se envían en lotes de 10 para no hacer una llamada por fragmento y mostrar el progreso. Los vectores quedan en una lista en memoria, que cumple el papel de la base de datos vectorial del diagrama de la guía.

3.3 Búsqueda semántica: buscar()

La pregunta del usuario se convierte en un vector con el mismo modelo de embeddings (debe ser el mismo para que los vectores estén en el mismo espacio). Luego se compara contra cada chunk con el producto punto:

```
parecido = np.dot(vector_pregunta, embeddings[i])
...
resultados.sort(reverse=True, key=lambda resultado: resultado[0])
return resultados[:3]
```

La guía define la similitud de coseno como $A \cdot B / (\|A\| \|B\|)$. Ollama entrega los embeddings normalizados (magnitud 1), así que el denominador vale 1 y el producto punto es exactamente la similitud de coseno. Los resultados se ordenan de mayor a menor similitud y se toman los 3 más cercanos.

3.4 Generación: preguntar_a_qwen()

Los 3 chunks recuperados, con su fuente, se concatenan en un bloque de contexto y se arma un prompt con instrucciones explícitas: responder **solo** con la información de los manuales, decir si la respuesta no aparece y citar archivo y página. Este prompt se envía a **qwen2.5:7b** con `ollama.chat()`. Estas instrucciones son las que reducen las alucinaciones: el modelo no responde de memoria sino a partir de texto real de los manuales.

```
mensaje = f"""
Responde usando solamente la informacion de los manuales.
Si la respuesta no aparece en los manuales, indicalo.
Menciona el nombre del archivo y la pagina utilizada.
Informacion de los manuales:
{contexto}
Pregunta:
{pregunta}
"""
```

3.5 Programa principal

Al iniciar, el programa lee los manuales y calcula todos los embeddings una sola vez (si no hay PDF, termina). Después entra en un ciclo `while True`: pide una pregunta, ejecuta `buscar()` y `preguntar_a_qwen()`, imprime la respuesta y repite hasta que el usuario escriba *salir*.

4. Ejercicio 3 — Chatbot con los documentos del curso

El código es el mismo pipeline RAG del ejercicio 2. La única diferencia es la carpeta de donde se leen los PDF: IA/ en lugar de manuales/. Esto muestra una ventaja de RAG: **el mismo programa sirve para cualquier dominio** solo cambiando los documentos, sin reentrenar el modelo. Es la idea del asistente virtual de la sección 7.7.1 de la guía (por ejemplo, un asistente para la Universidad).

	Ejercicio 2	Ejercicio 3
Carpeta de documentos	manuales/	IA/
Contenido	Manuales técnicos de máquinas de planta	Guías del curso IA y Minirobots
Modelos, chunks, top-k, prompt	Iguales	Iguales

5. Ejercicio 4 — Agente de mantenimiento

La guía (§7.8) dice que un agente necesita un modelo de IA, memoria y herramientas. Aquí el modelo es **qwen2.5:7b** y las herramientas son funciones de Python que consultan y modifican una base de datos SQLite (datos.db). El LLM no accede directamente a la base de datos: decide *qué herramienta usar y con qué argumentos*, y es Python quien la ejecuta.

5.1 Base de datos

`base_de_datos()` crea tres tablas (si no existen) y `repuestos()` carga el inventario inicial del robot ABB IRB140. Se usa `INSERT OR IGNORE` para que al reiniciar el programa no se dupliquen los repuestos.

Tabla	Columnas	Uso
repuestos	codigo (PK), nombre, equipo, cantidad, ubicacion	Catálogo e inventario del almacén
orden	numero (autoincremental), equipo, falla, fecha, estado	Órdenes de trabajo generadas
historial	numero, equipo, falla, fecha, orden	Registro de fallas por equipo, enlazado a la orden

Código	Repuesto	Cantidad	Ubicación
ABB-101	Batería SMB	0	B-50
ABB-102	Mazo de cables internos	5	B-55
ABB-103	Servomotor-CA	8	B-60
ABB-104	Correas dentadas de transmisión	20	B-65

La batería ABB-101 tiene cantidad 0 a propósito, para probar que el agente reporta correctamente un repuesto agotado en lugar de inventar existencias.

5.2 Herramientas y requisitos de la guía

Requisito de la guía	Herramienta	Qué hace
Buscar un repuesto (código)	buscar_repuesto(id)	Devuelve código, nombre y equipo. Convierte el código a mayúsculas para aceptar 'abb-101'.
(apoyo)	buscar(nombre)	Búsqueda por nombre con LIKE '%texto%', sin distinguir mayúsculas. Útil cuando el técnico dice 'el servomotor' sin saber el código.
Consultar en el almacén (código)	almacen(id)	Devuelve nombre, ubicación y cantidad disponible.
Generar orden de trabajo (falla, equipo)	orden(equipo, falla)	Inserta la orden con fecha y estado, obtiene su número con lastrowid y llama a actualizar_historial().
Actualizar el historial de fallas	actualizar_historial(...)	Se ejecuta automáticamente dentro de orden(); no se expone al LLM.
(apoyo)	consultar_historial(equipo)	Lista las fallas previas del equipo, de la más reciente a la más antigua, con su número de orden OT-XXXX.

Una decisión de diseño importante: `actualizar_historial()` **no** es una herramienta del LLM, sino que la llama `orden()`. Así es imposible crear una orden sin registrarla en el historial, aunque el modelo se olvide de hacerlo. Además todas las consultas SQL usan parámetros (?) en lugar de concatenar texto, lo que evita inyección SQL.

5.3 Cómo el LLM conoce las herramientas (tool calling)

Las funciones se pasan directamente a `ollama.chat(..., tools=herramientas)`. La librería de Ollama lee el nombre de cada función, sus *type hints* (`id: str`) y su *docstring*, y con eso genera la descripción que recibe el modelo. Por eso cada herramienta tiene un *docstring* con la sección *Args*: y un ejemplo ("*por ejemplo ABB-101*"): es la documentación que el LLM lee para decidir cuál usar.

```
def almacen(id: str) -> str:
    """
    Consulta la cantidad y ubicación de un repuesto.
    Args:
        id: Código del repuesto, por ejemplo ABB-101.
    """
```

5.4 Funcionamiento de agente()

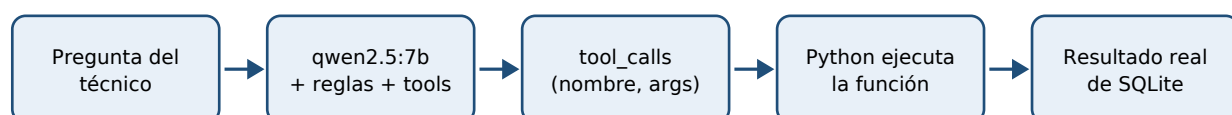


Figura 3. Ciclo del agente. El LLM elige la herramienta; los datos siempre salen de la base de datos.

- Se arma el mensaje con un **system prompt** de 11 reglas y la pregunta del usuario.
- Se llama al modelo con `temperature=0` para que sus decisiones sean deterministas y repetibles (no creativas).
- Si el modelo **no** pidió ninguna herramienta, el agente responde que no consultó la base de datos y pide el código o el nombre del repuesto, en vez de devolver una respuesta que el modelo pudo haber inventado.
- Si pidió herramientas, se recorren las `tool_calls`; el diccionario funciones traduce el nombre (texto) a la función real de Python, que se ejecuta con `funcion(**argumentos)`. Si el modelo pide una herramienta que no existe, se responde con un mensaje de error controlado.
- Se devuelven los resultados de las herramientas tal cual salen de la base de datos.

Por qué se devuelven los resultados directamente

Durante las pruebas, el modelo de 7B tendía a inventar cantidades o a escribir frases como "[Consultando inventario...]" sin haber consultado nada. Por eso las reglas se fueron agregando de forma iterativa (reglas 1, 2 y 8) y, además, la respuesta final no la redacta el LLM sino que es la salida real de las herramientas. Así el modelo solo actúa como **enrutador**: interpreta la intención del técnico y elige la acción, pero nunca es la fuente de los datos. Las reglas 3-6, 9 y 10 indican explícitamente qué herramienta usar en cada caso, lo cual mejora la precisión de un modelo pequeño.

Ejemplo de interacción esperada

Usuario: ¿Hay baterías ABB-101 en el almacén?

Agente:

Nombre: Batería SMB Ubicación: B-50 Cantidad: 0

Usuario: El ABB IRB140 tiene una falla en el servomotor del eje 2, genera la orden

Agente:

Orden N:0001 creada

Equipo: ABB IRB140

Fecha: Tuesday/Oct/06 | 12:31 PM

Falla: falla en el servomotor del eje 2

Estado: OK

6. Ejercicio 5 — LLM en un ESP32

Sí es posible, con matices. Hay dos escenarios:

- **Modelo local en el microcontrolador:** existen modelos de lenguaje ultracompactos que pueden correr directamente en el chip. Las variantes más adecuadas son el **ESP32-S3** y el **ESP32-P4**, que tienen instrucciones vectoriales (aceleran las multiplicaciones de matrices de las capas de atención y feedforward), PSRAM externa y memoria flash de hasta 16 MB para almacenar los pesos.
- **ESP32 como puente IoT:** el microcontrolador captura datos o la voz del usuario y los envía por Wi-Fi a un LLM grande en la nube o en un servidor local (por ejemplo, Ollama en un PC), y recibe la respuesta.

Restricciones del modelo local, relacionadas con la guía:

- **Memoria:** los modelos fundamentales tienen miles de millones de parámetros (§7.4); en un ESP32 solo cabe un modelo de muy pocos parámetros, limitado por la flash y la PSRAM.
- **Velocidad:** cada token generado requiere recorrer todo el modelo, así que la generación de texto es lenta.
- **Ventana de contexto corta:** la memoria disponible limita cuántos tokens puede mantener a la vez (§7.5, longitud del contexto).
- **Capacidad limitada:** un modelo tan pequeño solo sirve para tareas muy acotadas (comandos simples, clasificación de texto corto), no para razonamiento general.
- **En el modo puente,** la limitación pasa a ser la conectividad: latencia de red, dependencia de internet y consumo energético del Wi-Fi.

7. Cómo ejecutar los ejercicios 2, 3 y 4

```
git clone https://github.com/Jruiztr28/IA-y-Minirobots.git
cd IA-y-Minirobots/"Tarea 7"
pip install -r requirements.txt
ollama pull qwen2.5:7b
ollama pull nomic-embed-text
python <script_del_ejercicio>.py
```

Los scripts usan `Path(__file__).parent` para encontrar las carpetas manuales/, IA/ y el archivo `datos.db` relativos a su propia ubicación; por eso deben ejecutarse como archivos `.py` y no desde una celda de Colab (donde `__file__` no existe).

8. Limitaciones conocidas

- Los embeddings se recalculan cada vez que inicia el chatbot; con muchos PDF el arranque es lento. Se podrían guardar en disco o en una base vectorial (p. ej. ChromaDB).
- El corte de chunks es por número fijo de caracteres, por lo que una idea puede quedar partida entre dos chunks. Un solapamiento entre fragmentos mejoraría la recuperación.
- El agente no conserva memoria entre preguntas: cada consulta es independiente (la guía menciona la memoria como tercer componente de un agente).
- La orden de trabajo se crea con estado fijo y no hay una herramienta para cerrarla o cambiar su estado.